

Lens执行器控制器使用指南

版本：0.1.0-alpha

本文档整合了Lens执行器的API接口说明和配置文件格式，为开发者提供完整的使用指导。

1. 接口概述

`ActuatorController`是一个用于控制执行器的C++类，提供了初始化执行器、设置目标值（位置、速度、扭矩、电流等）以及获取执行器当前状态的功能。该类支持单执行器控制和多执行器批量控制，适用于基于CAN总线和EtherCAT通信协议的执行器控制系统。

2. 头文件与依赖

使用前需包含头文件：

```
#include "actuator_controller.h"
```

依赖项：

- C++标准库：`vector`、`string`
- 内部使用私有实现类 `ActuatorControllerPrivate`
- 配置文件解析：使用 `nlohmann/json` 库解析JSON配置文件
- 日志系统：基于 `spdlog` 库实现，需要安装spdlog库（ROS2环境中默认已安装）

3. Lens执行器配置文件说明

`ActuatorController` 使用两个重要的配置文件来设置Lens执行器参数和连接信息。

3.1 执行器连接配置文件 (actuator_config.json)

此文件用于配置Lens执行器的连接信息和基本参数。

根结构

```
{
  "acutator_list": [
    // 从站配置列表
  ]
}
```

注意：根字段名存在拼写错误，应为 `actuator_list`，但 `JsonParser` 实现中仍使用 `acutator_list`

从站配置结构

每个从站配置包含以下字段：

字段名	类型	是否必需	默认值	说明
ethercat_slave_id	整数	是	-	EtherCAT从站ID
boardcast	布尔值	否	false	是否启用广播模式
can_id_list	数组	是	-	连接到此从站的执行器配置列表

Lens执行器配置结构 (can_id_list 中的元素)

字段名	类型	是否必需	默认值	说明
can_id	整数	是	-	执行器的CAN ID，同一从站同一端口下必须唯一
port	整数	是	-	执行器连接的端口号
type	字符串	是	-	执行器类型，此处应为 "lens"
model	字符串	否	"default"	Lens执行器具体型号，目前仅有"default"型号
joint_name	字符串	是	-	关节名称，全局必须唯一且不能为空
mode	字符串	否	"mit"	执行器模式，支持的 值："mit"、"position"、"profile_position"

Lens执行器配置示例

```
{
  "acutator_list": [
    {
      "ethercat_slave_id": 1,
      "boardcast": false,
      "can_id_list": [
        {
          "can_id": 1,
          "joint_name": "lens_joint_1",
          "port": 1,
          "type": "lens",
          "model": "default"
        }
      ]
    }
  ]
}
```

3.2 执行器参数配置文件 (actuator_params_config.json)

此文件用于配置Lens执行器的参数范围限制。

根结构

```
{
  "actuator_params": {
    "lens": {
      "default": {
        "pos_lower": -6.28,
        "pos_upper": 6.28,
        "vel_lower": -10.0,
        "vel_upper": 10.0,
        "kp_range": 200.0,
        "kd_range": 2.0,
        "tor_lower": -15.0,
        "tor_upper": 15.0
      }
    }
  }
}
```

参数范围说明

每个型号配置包含以下参数范围：

字段名	类型	说明
pos_lower	浮点数	位置下限（单位：弧度）
pos_upper	浮点数	位置上限（单位：弧度）
vel_lower	浮点数	速度下限（单位：弧度/秒）
vel_upper	浮点数	速度上限（单位：弧度/秒）
kp_range	浮点数	位置增益Kp的范围值
kd_range	浮点数	阻尼增益Kd的范围值
tor_lower	浮点数	扭矩下限（单位：Nm）
tor_upper	浮点数	扭矩上限（单位：Nm）

4. 类的构造与析构

4.1 构造函数

```
ActuatorController();
```

创建一个ActuatorController实例，用于后续的执行器控制操作。

4.2 析构函数

```
~ActuatorController();
```

释放ActuatorController实例占用的资源。

5. 初始化方法

5.1 初始化CAN ID和EtherCAT从站关系

```
bool initCanIdAndEthercatRelation(const std::string& ifname, const std::string&
config_file, int send_frequency = 500);
```

功能：初始化CAN ID和EtherCAT从站的映射关系，建立与Lens执行器的通信连接。函数内部已有重试机制，且默认直接进入OP状态（运行状态）。初始化成功后，**必须调用使能电机函数**才能控制关节。

参数：

- **ifname**：网络接口名称（例如"eth0"）
- **config_file**：JSON配置文件路径，包含CAN ID和EtherCAT从站的映射关系
- **send_frequency**：EtherCAT发送频率，单位Hz，默认500Hz

返回值：成功返回true，失败返回false。

5.2 加载执行器参数配置

```
static bool loadActuatorParamsConfig(const std::string& params_file =
std::string("./actuator_params_config.json"));
```

功能：加载Lens执行器的参数配置文件，设置执行器的运行参数。**此静态函数必须在创建ActuatorController对象之前调用**，否则将自动加载默认参数配置。

参数：

- **params_file**：执行器参数配置文件路径，默认为"./actuator_params_config.json"

返回值：成功返回true，失败返回false。

6. 日志系统控制

6.1 设置日志输出级别

```
static void setLogLevel(int level);
```

功能：设置日志输出级别，控制日志详细程度。

参数：

- `level`：日志级别 (0-TRACE, 1-DEBUG, 2-INFO, 3-WARNING, 4-ERROR, 5-CRITICAL)

6.2 设置日志输出到文件

```
static void setLogFile(const std::string& log_file, bool truncate = false);
```

功能：设置日志输出到指定文件，或仅输出到控制台。

参数：

- `log_file`：日志文件路径，为空则只输出到控制台
- `truncate`：是否清空已有文件，默认为false（追加模式）

7. Lens执行器控制方法

7.1 使能电机

单执行器版本：

```
int enableMotor(const std::string& joint_name);
```

批量版本：

```
int enableMotor(const std::vector<std::string>& joint_names);
```

功能：使能Lens执行器电机，使执行器可以接收控制指令。初始化后必须先调用此函数，执行器才能正常工作。

参数：

- `joint_name/joint_names`：Lens执行器的关节名称

返回值：成功返回0，失败返回非0值。

7.2 禁用电机

单执行器版本：

```
int disableMotor(const std::string& joint_name);
```

批量版本：

```
int disableMotor(const std::vector<std::string>& joint_names);
```

功能：禁用Lens执行器电机，使执行器停止工作。

参数：

- `joint_name/joint_names`：Lens执行器的关节名称

返回值：成功返回0，失败返回非0值。

7.3 设置电机模式

单执行器版本：

```
int setMotorMode(const std::string& joint_name, MotorModeEnum mode);
```

批量版本：

```
int setMotorMode(const std::vector<std::string>& joint_names, const  
std::vector<MotorModeEnum>& modes);
```

功能：设置Lens执行器的工作模式。目前仅支持**MIT模式**、**位置模式**和**梯形位置模式**。

注意事项：设置模式前关节电机必须处于失能状态，设置模式后需要重新使能电机才能开始控制关节。

参数：

- `joint_name/joint_names`：Lens执行器的关节名称
- `mode/modes`：电机模式枚举值

返回值：成功返回0，失败返回非0值。

7.4 设置MIT参数控制

单执行器版本：

```
int setTargetMit(const std::string& joint_name, double position, double velocity,  
double torque, double kp, double kd);
```

批量版本：

```
int setTargetMit(const std::vector<std::string>& joint_names, const
std::vector<double>& positions, const std::vector<double>& velocities, const
std::vector<double>& torques, const std::vector<double>& kps, const
std::vector<double>& kds);
```

功能：设置Lens执行器的位置、速度、扭矩目标值以及PID控制参数。执行器必须处于**MIT**模式。

参数：

- `joint_name/joint_names`：Lens执行器的关节名称
- `position/positions`：目标位置
- `velocity/velocities`：目标速度
- `torque/torques`：目标扭矩
- `kp/kps`：比例增益参数
- `kd/kds`：微分增益参数

返回值：成功返回0，失败返回非0值。

7.5 移动到零点位置

单执行器版本：

```
int moveToZero(const std::string& joint_name);
```

批量版本：

```
int moveToZero(const std::vector<std::string>& joint_names);
```

功能：将Lens执行器移动到零点位置。执行器必须处于位置模式或梯形位置模式。

参数：

- `joint_name/joint_names`：Lens执行器的关节名称

返回值：成功返回0，失败返回非0值。

7.6 设置目标位置

单执行器版本：

```
int setTargetPosition(const std::string& joint_name, double target_position);
```

批量版本：

```
int setTargetPosition(const std::vector<std::string>& joint_names, const
std::vector<double>& target_positions);
```

功能：设置Lens执行器的目标位置。执行器必须处于位置模式或梯形位置模式。

参数：

- `joint_name/joint_names`：Lens执行器的关节名称
- `target_position/target_positions`：目标位置值

返回值：成功返回0，失败返回非0值。

8. 状态获取方法

8.1 获取当前位置

单执行器版本：

```
int getCurrentPosition(const std::string& joint_name, double &current_position);
```

批量版本：

```
int getCurrentPosition(const std::vector<std::string>& joint_names,
std::vector<double>& current_positions);
```

功能：获取Lens执行器的当前位置。

参数：

- `joint_name/joint_names`：Lens执行器的关节名称
- `current_position/current_positions`：用于存储当前位置的变量引用/向量

返回值：成功返回0，失败返回非0值。

8.2 获取当前速度

单执行器版本：

```
int getCurrentVelocity(const std::string& joint_name, double &current_velocity);
```

批量版本：

```
int getCurrentVelocity(const std::vector<std::string>& joint_names,
std::vector<double>& current_velocities);
```


功能：获取Lens执行器的当前速度。

参数：

- `joint_name/joint_names`：Lens执行器的关节名称
- `current_velocity/current_velocities`：用于存储当前速度的变量引用/向量

返回值：成功返回0，失败返回非0值。

8.3 获取当前扭矩

单执行器版本：

```
int getCurrentTorque(const std::string& joint_name, double &current_torque);
```

批量版本：

```
int getCurrentTorque(const std::vector<std::string>& joint_names,  
std::vector<double>& current_torques);
```

功能：获取Lens执行器的当前扭矩。

参数：

- `joint_name/joint_names`：Lens执行器的关节名称
- `current_torque/current_torques`：用于存储当前扭矩的变量引用/向量

返回值：成功返回0，失败返回非0值。

8.4 获取当前电流

单执行器版本：

```
int getCurrentCurrent(const std::string& joint_name, double &current_current);
```

批量版本：

```
int getCurrentCurrent(const std::vector<std::string>& joint_names,  
std::vector<double>& current_currents);
```

功能：获取Lens执行器的当前电流。

参数：

- `joint_name/joint_names` : Lens执行器的关节名称
- `current_current/current_currents` : 用于存储当前电流的变量引用/向量

返回值：成功返回0，失败返回非0值。

9. 使用示例

9.1 Lens执行器初始化与基本操作示例

```
#include <iostream>
#include <string>
#include "actuator_controller.h"

int main() {
    std::cout << "==== Lens Actuator Controller Test Program =====" <<
    std::endl;

    // 0. 加载执行器参数配置 ( 必须在创建ActuatorController对象之前调用 )
    // 配置文件格式详见 config_file_formats.md 文档
    if
(!ActuatorController::loadActuatorParamsConfig("./actuator_params_config.json")) {
        std::cerr << "Failed to load actuator parameters configuration." <<
    std::endl;
        return -1;
    }
    std::cout << "Lens actuator parameters loaded successfully." << std::endl;

    // 1. 创建ActuatorController实例
    ActuatorController controller;

    // 2. 初始化参数
    // 执行器连接配置文件路径，格式详见 config_file_formats.md 文档
    std::string config_file = "./actuator_config.json";
    std::string ifname = "eth0";

    // 3. 初始化CAN ID和EtherCAT关系 ( 函数内部已有重试机制 )
    if (controller.initCanIdAndEthercatRelation(ifname, config_file, 500)) {
        std::cout << "Lens Actuator Controller initialized successfully." <<
    std::endl;
    } else {
        std::cerr << "Failed to initialize Lens Actuator Controller." <<
    std::endl;
        return -1;
    }

    // 4. 使能电机 ( 初始化后必须执行此步骤 )
    std::string joint_name = "lens_joint_1";
    if (controller.enableMotor(joint_name) == 0) {
        std::cout << "Successfully enabled motor for joint '" << joint_name << "'
    << std::endl;
    } else {
        std::cerr << "Failed to enable motor for joint '" << joint_name << "' <<
```

```

std::endl;
    return -1;
}

// 5. 设置电机模式 ( 需要先失能 , 再设置模式 , 最后重新使能 )
double position = 0.5;
double velocity = 0.1;
double torque = 0.0;
double kp = 10.0;
double kd = 1.0;

// 设置模式前确保电机处于失能状态
controller.disableMotor(joint_name);

// 设置执行器为MIT模式
if (controller.setMotorMode(joint_name, MODE_MIT) == 0) {
    std::cout << "Successfully set motor mode to MIT for joint '" <<
joint_name << "'" << std::endl;
} else {
    std::cerr << "Failed to set motor mode for joint '" << joint_name << "'"
<< std::endl;
    return -1;
}

// 重新使能电机
if (controller.enableMotor(joint_name) == 0) {
    std::cout << "Successfully re-enabled motor for joint '" << joint_name <<
"" << std::endl;
} else {
    std::cerr << "Failed to re-enable motor for joint '" << joint_name << "'"
<< std::endl;
    return -1;
}

// 设置目标值
if (controller.setTargetMit(joint_name, position, velocity, torque, kp, kd) ==
0) {
    std::cout << "Successfully set target for Lens actuator." << std::endl;
} else {
    std::cerr << "Failed to set target for Lens actuator." << std::endl;
}

// 6. 获取执行器当前状态
double current_pos, current_vel, current_torque, current_current;

if (controller.getCurrentPosition(joint_name, current_pos) == 0) {
    std::cout << "Current position: " << current_pos << std::endl;
}

if (controller.getCurrentVelocity(joint_name, current_vel) == 0) {
    std::cout << "Current velocity: " << current_vel << std::endl;
}

if (controller.getCurrentTorque(joint_name, current_torque) == 0) {

```

```

        std::cout << "Current torque: " << current_torque << std::endl;
    }

    if (controller.getCurrentCurrent(joint_name, current_current) == 0) {
        std::cout << "Current current: " << current_current << std::endl;
    }

    // 7. 禁用电机 ( 程序结束前可选 )
    controller.disableMotor(joint_name);

    return 0;
}

```

9.2 多Lens执行器批量控制示例

```

#include <iostream>
#include <vector>
#include "actuator_controller.h"

int main() {
    // 0. 加载执行器参数配置 ( 必须在创建ActuatorController对象之前调用 )
    if (!ActuatorController::loadActuatorParamsConfig()) {
        std::cerr << "Failed to load Lens actuator parameters configuration." <<
std::endl;
        return -1;
    }

    // 1. 创建控制器实例
    ActuatorController controller;

    // 2. 初始化
    std::string config_file = "./actuator_config.json";
    std::string ifname = "eth0";
    if (!controller.initCanIdAndEthercatRelation(ifname, config_file)) {
        std::cerr << "Failed to initialize Lens Actuator Controller." <<
std::endl;
        return -1;
    }

    // 3. 准备多个Lens执行器的参数
    std::vector<std::string> joint_names = {"lens_joint_1", "lens_joint_2",
"lens_joint_3"};

    // 4. 批量使能电机
    if (controller.enableMotor(joint_names) == 0) {
        std::cout << "Successfully enabled motors for all " << joint_names.size()
<< " Lens actuators" << std::endl;
    } else {
        std::cerr << "Failed to enable motors for all Lens actuators" <<
std::endl;
        return -1;
    }
}

```

```

    }

    // 5. 批量设置电机模式和MIT参数
    std::vector<double> positions = {0.5, 0.3, 0.1};
    std::vector<double> velocities = {0.1, 0.05, 0.0};
    std::vector<double> torques = {0.0, 0.0, 0.0};
    std::vector<double> kps = {10.0, 8.0, 5.0};
    std::vector<double> kds = {1.0, 0.8, 0.5};

    // 批量设置模式前先失能所有电机
    controller.disableMotor(joint_names);

    // 确保执行器处于MIT模式
    std::vector<MotorModeEnum> modes(joint_names.size(), MODE_MIT);
    if (controller.setMotorMode(joint_names, modes) == 0) {
        std::cout << "Successfully set motor modes to MIT for all Lens actuators"
<< std::endl;
    } else {
        std::cerr << "Failed to set motor modes for all Lens actuators" <<
std::endl;
        return -1;
    }

    // 重新批量使能电机
    if (controller.enableMotor(joint_names) == 0) {
        std::cout << "Successfully re-enabled motors for all Lens actuators" <<
std::endl;
    } else {
        std::cerr << "Failed to re-enable motors for all Lens actuators" <<
std::endl;
        return -1;
    }

    // 批量设置目标值
    if (controller.setTargetMit(joint_names, positions, velocities, torques, kps,
kds) == 0) {
        std::cout << "Successfully set targets for multiple Lens actuators." <<
std::endl;
    } else {
        std::cerr << "Failed to set targets for multiple Lens actuators." <<
std::endl;
    }

    // 6. 获取多个执行器的状态
    std::vector<double> current_positions, current_velocities, current_torques,
current_currents;
    if (controller.getCurrentPosition(joint_names, current_positions) == 0 &&
        controller.getCurrentVelocity(joint_names, current_velocities) == 0 &&
        controller.getCurrentTorque(joint_names, current_torques) == 0 &&
        controller.getCurrentCurrent(joint_names, current_currents) == 0) {

        for (size_t i = 0; i < joint_names.size(); ++i) {
            std::cout << "\nStatus of Lens actuator '" << joint_names[i] << "':"
<< std::endl;

```

```
        std::cout << "   Position: " << current_positions[i] << std::endl;
        std::cout << "   Velocity: " << current_velocities[i] << std::endl;
        std::cout << "   Torque: " << current_torques[i] << std::endl;
        std::cout << "   Current: " << current_currents[i] << std::endl;
    }
}

// 7. 批量禁用电机
controller.disableMotor(joint_names);

return 0;
}
```

10. 错误处理最佳实践

1. **操作结果检查**：所有控制方法和状态获取方法都返回整数值，应始终检查返回值以确保操作成功。
2. **配置文件路径验证**：确保提供的配置文件路径正确，文件存在且格式符合要求。
3. **电机使能检查**：初始化后必须先调用使能电机函数，执行器才能正常响应控制指令。
4. **模式兼容性**：在设置目标值前，确保执行器处于正确的工作模式（MIT模式、位置模式或梯形位置模式）。
5. **模式设置流程**：设置电机模式前必须先使电机处于失能状态，设置完成后需要重新使能电机才能开始控制。

11. 注意事项

1. 所有接口方法都支持通过关节名称(joint_name)进行单执行器操作和批量操作（通过向量），根据实际需求选择合适的接口。
2. 使用前必须正确初始化控制器，否则后续操作可能失败。
3. 初始化后必须调用enableMotor函数使能电机，执行器才能接收控制指令。
4. 目前仅支持MIT模式、位置模式和梯形位置模式。
5. loadActuatorParamsConfig静态函数必须在创建ActuatorController对象之前调用，否则将自动加载默认参数配置。
6. 确保关节名称与实际执行器对应，避免错误控制。
7. 类的拷贝构造和赋值操作已被禁用，确保正确使用实例。
8. 初始化函数内部已有重试机制，无需在外部实现重试逻辑。
9. 程序安全重启：机器人在控制中途程序退出时，重新启动控制程序前，应先断电确保机器人处于安全姿态，再进行下一步控制操作。